

Pixelated Dissolve Shader

Victoria Dunn

University Of Southern California

July 2026

Abstract

This paper will discuss the journey of creating a TRON-styled dissolve shader in Unity's URP. We will take a dive into the challenges I faced when developing this shader as well as how to solve these issues.

1 Translating HDRP into URP

The topic for this directed research was to create a Tron-style pixelated dissolver shader in Unity's URP. The inspiration for this was a Twitter thread by Mirza Beig, who created a similar shader, but in HDRP instead. Figuring out how to translate a shader from HDRP to URP was moderately difficult, but once I got the hang of moving between the two, it became easier. Overall, the two are similar, but there are a few differences that need to be figured out: some setup is different, and more importantly, some nodes are not available in URP. For instance, URP does not have an append node; instead, it has add and combine nodes that essentially do the same thing.

First off, when creating the dissolver, I ignored the pixelation portion and focused on getting the shader to dissolve. After looking at the example and doing some research, I created the dissolve mask by using a simple noise node with two variables: noise scale and noise strength, which controlled how strong a noise was generated. For the mask, I needed to subtract a dissolve threshold variable (decides what stays and what gets dissolved), edit the value ranges (one minus and step nodes), and connect to the Alpha input of the fragment node.

Next, I incorporated quantization in the dissolver so that the dissolver would have a pixelated effect, rather than a smooth dissolve effect. I used the formula $\text{floor}(\text{value} * \text{steps}) / \text{steps}$, where the value was the world position of the object, and the steps variable was the size of the goal pixel. This was a pretty straightforward translation from HDRP to URP. However, the problem I faced was trying to figure out the pipeline between the quantized pixels and adding the transition glow band.

The transition glow band was by far the most challenging to figure out in URP. My biggest struggle with this was trying to understand what components were needed. For example, which pieces of the whole shader graph were relevant, and which needed to be separate. My initial understanding was that everything needed to be connected through the quantized coordinates that I created, so that everything would be pixelated properly and equally. However, this was not the case. Following the HDRP outline for this was a bit confusing. When looking at the shader graph and the individual variables, it didn't click right away that there needed to be two different masks: one for the dissolve and another for the glow portion. After some time looking at the HDRP graph and variables, I found that subtracting these two masks would generate the transition band that would then be used for the emission portion of the material. Additionally, I found that the second mask needed to have its own coordinates and needed to generate its own separate grid. To do this, I used the tiling and offset node to create tiles, the fraction node for creating multiple tiles at a time, and the rectangle node so that we would have equal-sized squares. This was the simplest solution I found to solve the grid pattern that was missing for the transition band. The pipeline for this in URP was pretty similar to HDRP, again with a few modifications of the nodes being used.

2 Issues and Workarounds

Throughout my journey of this project, I faced several challenges. The obvious primary issue was taking the HDRP example and using it as a guideline for my URP shader. When it came to the early days of working on the dissolver, I had a small issue figuring out how to get it to start from the top and dissolve to the bottom, rather than the initial randomized dissolving. The random dissolve pattern was due to using a simple noise to generate the dissolve mask. I solved this issue after working on the pixelation portion of the dissolve mask. The solution was to extract the G (Y position) of the object and add that to the remapped noise before continuing with the normal pipeline. While working on the dissolve shader, another obstacle I came across was the pixelation. The good news was that the formula used in the HDRP example could easily be used in URP. The only thing I needed to do was figure out what the variables meant and how they would fit into my graph. For the pipeline of this shader graph, I started with quantizing the coordinates of the object, then used these for the dissolve mask (pixel size the dissolve effect and use the Y position so that the mask dissolves top to bottom).

The biggest issue I had to figure out was how to get the shader to emulate the actual pixelated glow effect. This issue had multiple layers the deeper I got into the project. The first issue was getting the transition band. My initial instinct was to use the quantized coordinates to create the band. I did this by using the Y position of the quantized coordinates, multiplying by a glow strength and a glow color, then plug into the emission section of the shader. This created a glowing line on the edge that got dissolved. However, this was not the desired outcome, since the goal was to have individual, pixelated “squares” appear whenever you dissolve a section of the object. So, to fix this, I did the following. I used completely different controls and created a grid that did not use the already quantized pixels I used for the dissolve mask. I made another mask, the edge glow mask, to subtract from the dissolve mask. This new mask consisted of the variables dissolve threshold (set what disappears and what appears), added to the edge width of the band, subtracted by the noise, used a step node to edit the range, and subtracted this from the dissolve mask. Note: subtracting the edge glow mask from the dissolve mask did not work as it should have in theory. Instead, subtracting the dissolve mask from this mask actually got the transition band to appear. Now that the transition band exists, there was a new problem to be solved: how to get the band to look pixelated. From the previous problem that was just fixed with trial and error, I learned that using the same quantized coordinates that were used for the dissolve mask would not cut it. So, I needed to generate a grid pattern that could be used to mimic the effect. This was a challenge since I was thinking too literally and thought that I needed to layer this grid on top of my texture or glow band. But the solution was the following: tiling and offset node to create tiles, the fraction node for creating multiple tiles at a time, and the rectangle node so that there are individual squares. This did not work at first since it did not appear, but after playing around with the controls, I switched the dissolve mask from being plugged into the Alpha to the Alpha Clip Threshold, and that did the trick. However, after this victory, two issues arose: the dissolving effect became inverted, and now it dissolved from the bottom to top, and the glow band started on the bottom. I first thought to use the one minus node on either the dissolve mask or transition band mask to get the values within the correct range for the dissolver, but just using it for one of the masks did not do the trick. Instead, I needed to use one minus after subtracting the dissolve threshold from the noise remap and another one minus after subtracting both masks.

3 Shader Graph Breakdown

This final section will break down the Dissolver Shader Graph and explain what each section in the pipeline does.

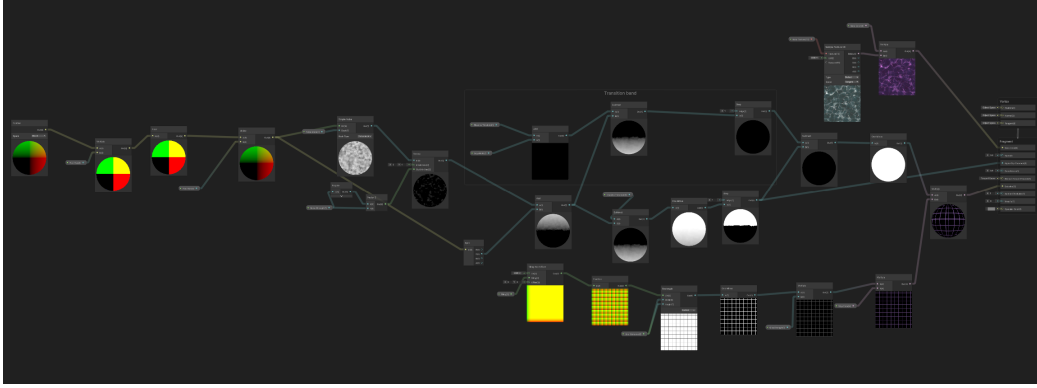


Figure 1: Whole Shader Graph

The variables that are used in this shader graph are the following: Base Color, Base Texture, Dissolve Threshold, Noise Scale, Noise Strength, Edge Width, Edge Color, Pixel Size, Glow Strength, Tiling, and Line Thickness. Base Color and Base Texture are the variables that are used for the material's color and texture. Dissolve Threshold determines how much of the mesh is visible. Noise Scale determines how much noise will be generated, and Noise Strength modifies the intensity of the noise. The closer to zero, the denser the noise will appear, creating a linear effect. The Edge Width variable modifies how thick the transition glow band is, and Edge Color modifies the color of the glow. Glow Strength is in charge of how strong the brightness of the glow is. Pixel Size, which edits the number of pixels that appear for the dissolve mask. Tiling determines the number of squares in the grid pattern for the transition glow band, and line thickness determines how thick the lines of the grid are.

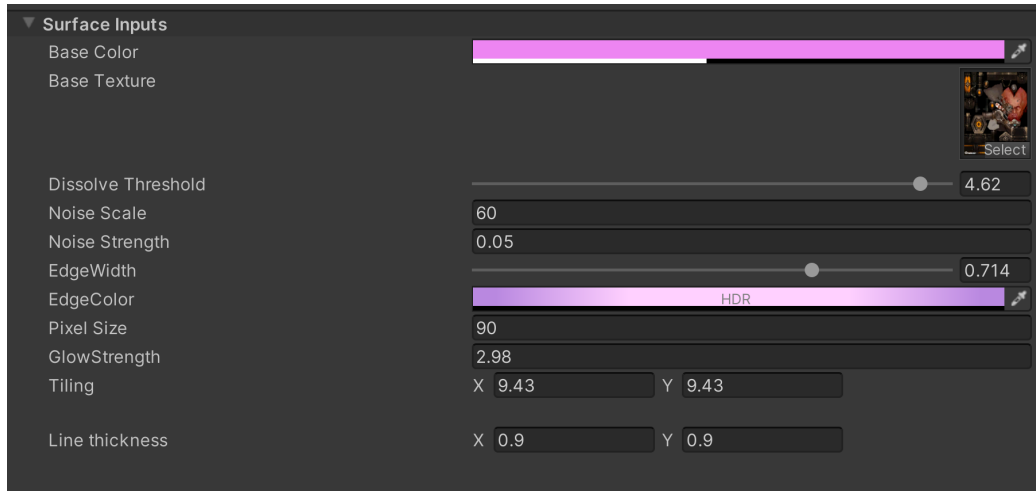


Figure 2: Exposed Properties/Variables

The figure below displays how to create pixels. I used the formula:

$$\text{floor}(\text{world_position_of_object} * \text{pixel_size}) / \text{pixel_size}. \quad (1)$$



Figure 3: Quantized Position

The next figure depicts where the noise is generated. This takes the quantized world position, put them into the UV port of a simple noise node, scales it with the Noise Scale, and output them to the remap node with the negate vector of the variable Noise Strength.

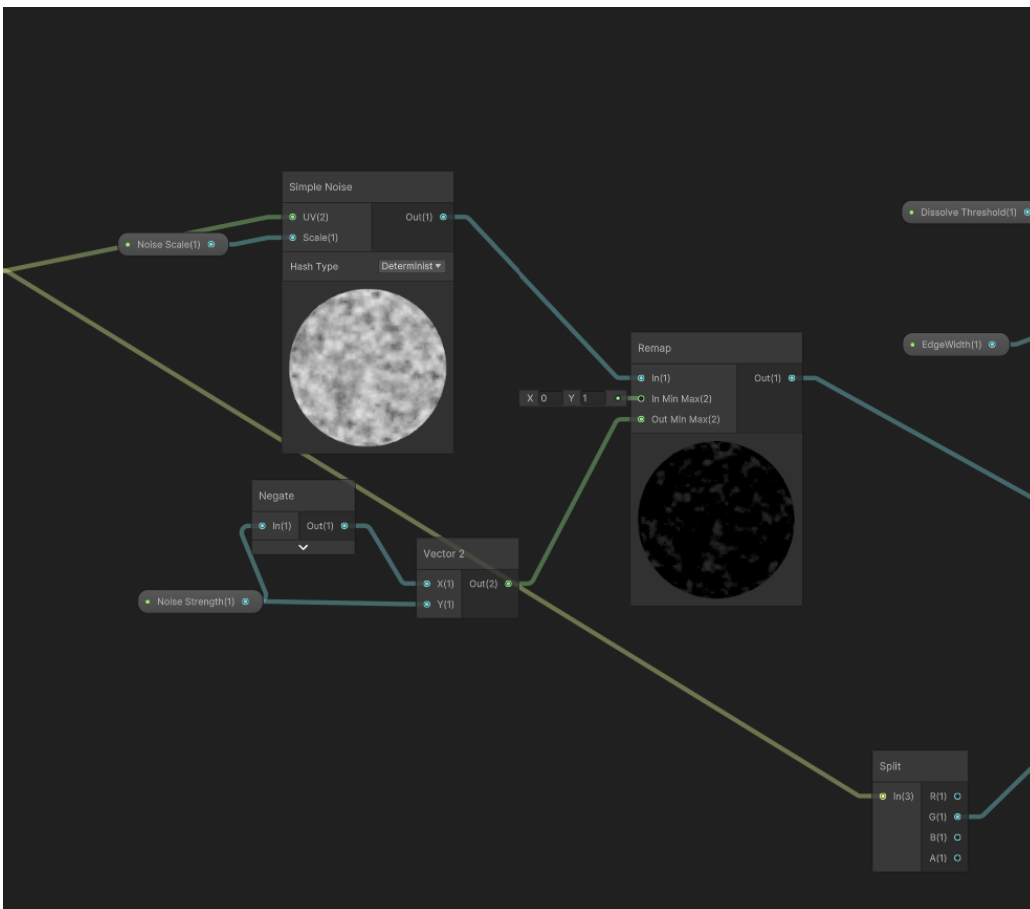


Figure 4: Noise Generator

The next section depicts the dissolve mask that is created. This part takes the remapped noise and adds the Y position of the object to it so that the dissolve will happen on the Y axis specifically.

This ensures that we can have the dissolve effect go from the top to the bottom. Next, we subtract the Dissolve Threshold from these coordinates to start dissolving the object. Then we use the one minus node on our result and use the Step node to convert our coordinates into binary. Then this gets outputted to the Alpha Clip Threshold.

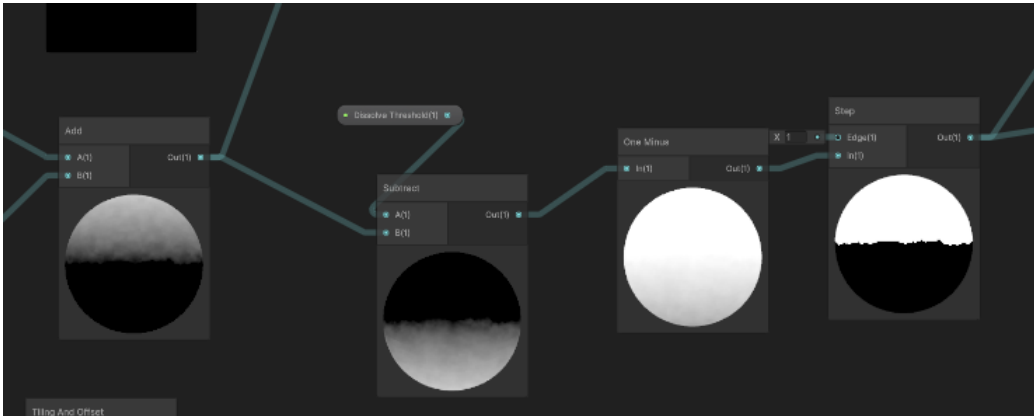


Figure 5: Dissolve Mask

Next, we have our other mask that creates the edge glow mask. This glow mask adds the Dissolve Threshold with the Edge Width, subtracts it from the quantized coordinate, converts to binary, then subtracts the dissolve mask from it, thus creating a new mask that will be used for the transition glow band. Usually the glow mask is subtracted from the dissolve mask; however, in this case, the reverse worked better for our desired outcome. Lastly, we use the One Minus node to stay in our desired range.

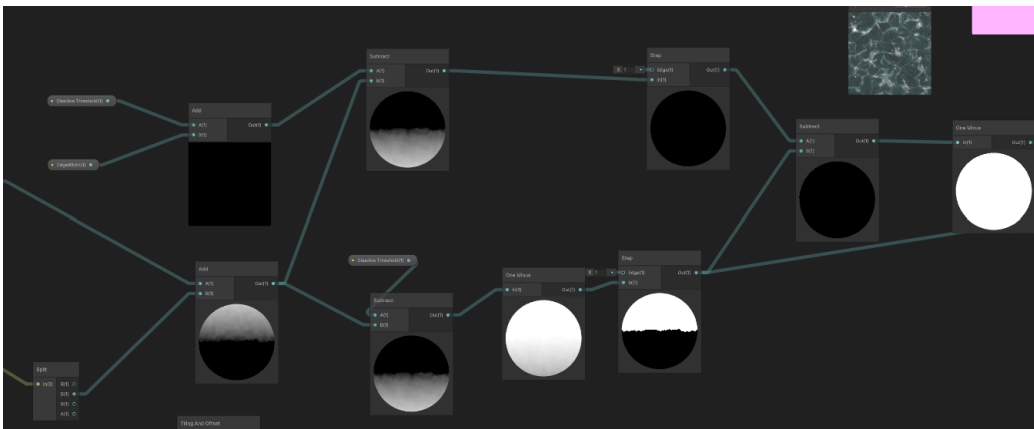


Figure 6: Edge Glow Mask

Adding on, we have the grid pattern of the transition glow band. This part creates the number of tiles that make up the grid pattern, creates squares, and adds the glow and color of the desired grid. This section is then multiplied by the new mask that is generated by the subtraction of the dissolve mask and the edge glow mask and plugged into the emission.

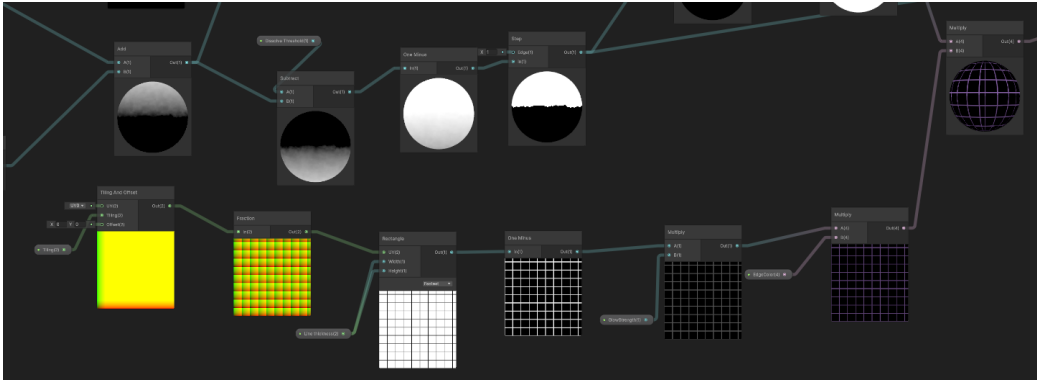


Figure 7: Glow Transition Band

The last section is the texture and base color of the material. Here we take a texture and multiply it by a color. Plug it into the color section of the output shader to create a base. Here you can play around with different textures and whether or not you want to change its color.

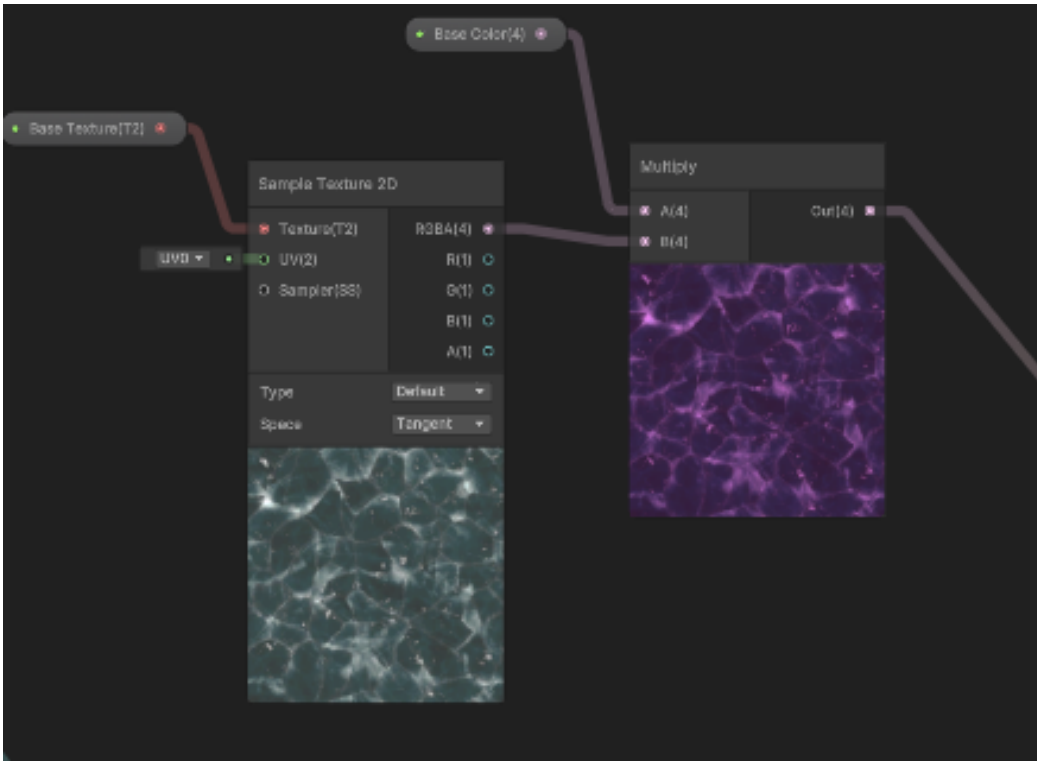


Figure 8: Base Texture and Color